

BERT (Bidirectional Encoder Representations from Transformers) is a powerful pre-trained language model developed by Google. Its key innovation lies in its bidirectional training approach, allowing it to understand the context of a word based on both the words that come before and after it in a sentence. This is a significant improvement over previous models that only considered one direction.

Here's a breakdown of its functionality and applications:

Functionality:

- **Pre-training:** BERT is initially pre-trained on a massive dataset of text (like Wikipedia and BookCorpus) using two unsupervised tasks:
 - **Masked Language Model (MLM):** Randomly masks out some words in the input and trains the model to predict the original words. This helps BERT learn to understand the context of words.
 - **Next Sentence Prediction (NSP):** Trains the model to predict whether two sentences are consecutive in the original text. This helps BERT understand the relationship between sentences.
- **Fine-tuning:** After pre-training, BERT can be fine-tuned on smaller, task-specific datasets for various downstream NLP tasks. This involves adding a simple output layer on top of the pre-trained BERT model and training it on the specific task.
- **Transformer Architecture:** BERT is based on the Transformer architecture, which uses attention mechanisms to weigh the importance of different words in the input sequence. This allows BERT to capture long-range dependencies in the text.

Applications:

BERT has achieved state-of-the-art results on a wide range of NLP tasks, including:

- **Question Answering:** Answering questions based on a given text passage.
- **Sentiment Analysis:** Determining the emotional tone of a text (positive, negative, neutral).
- **Text Classification:** Categorizing text into different classes (e.g., spam detection, topic classification).
- **Named Entity Recognition (NER):** Identifying and classifying named entities (like people, organizations, locations) in text.
- **Machine Translation:** Translating text from one language to another.
- **Summarization:** Generating concise summaries of longer texts.
- **Relation Extraction:** Identifying relationships between entities in text.

In essence, BERT's ability to deeply understand the context of language has revolutionized many NLP tasks, making it a foundational technology in the field.

Start coding or [generate](#) with AI.

To use a pre-trained BERT model, you'll typically use a library like `transformers` from Hugging Face. First, install the library:

```
!pip install transformers torch
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (4.56.1)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.8.0+cu126)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.19.1)
Requirement already satisfied: huggingface-hub<1.0,>=0.34.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.34.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2024.11.6)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers) (2.32.4)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.6.2)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.1)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.3)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch) (3.5)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.80)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.0.4)
Requirement already satisfied: nvidia-curand-cu12==10.3.7.7 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.7.7)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.1.2)
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.4.2 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.4.2)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.3 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.3)
Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.77)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in /usr/local/lib/python3.12/dist-packages (from torch) (12.6.85)
Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in /usr/local/lib/python3.12/dist-packages (from torch) (1.11.1.6)
Requirement already satisfied: triton==3.4.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0,>=0.34.0) (1.1.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch) (3.0.2)
```

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers)
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (3.10)
 Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (2.5)
 Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers) (202

Here's an example of how to use BERT for Question Answering:

```
from transformers import BertForQuestionAnswering, BertTokenizer
import torch

# Load a pre-trained BERT model and tokenizer for question answering
model = BertForQuestionAnswering.from_pretrained('bert-large-uncased-whole-word-masking-finetuned-squad')
tokenizer = BertTokenizer.from_pretrained('bert-large-uncased-whole-word-masking-finetuned-squad')

# Example text (context) and question
text = """
BERT (Bidirectional Encoder Representations from Transformers) is a powerful pre-trained language model developed by Google. It
has achieved state-of-the-art results on a wide range of NLP tasks, including Question Answering, Sentiment Analysis, Text
Classification, and more.

Question: Who developed BERT?
"""

# Tokenize the text and question
inputs = tokenizer(question, text, return_tensors='pt')

# Get the start and end logits
with torch.no_grad():
    outputs = model(**inputs)
    start_logits = outputs.start_logits
    end_logits = outputs.end_logits

# Get the predicted start and end positions
answer_start_index = torch.argmax(start_logits)
answer_end_index = torch.argmax(end_logits)

# Get the tokens from the input IDs
all_tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])

# Get the answer span
answer = tokenizer.convert_tokens_to_string(all_tokens[answer_start_index:answer_end_index+1])

print(f"Context: {text}")
print(f"Question: {question}")
print(f"Answer: {answer}")
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
 The secret `HF\_TOKEN` does not exist in your Colab secrets.
 To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as an environment variable.
 You will be able to reuse this secret in all of your notebooks.
 Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(
 config.json: 100% 443/443 [00:00<00:00, 34.4kB/s]
 model.safetensors: 100% 1.34G/1.34G [00:29<00:00, 72.7MB/s]
 Some weights of the model checkpoint at bert-large-uncased-whole-word-masking-finetuned-squad were not used when initializing BertForQuestionAnswering. This is expected if you are initializing BertForQuestionAnswering from the checkpoint of a model trained on another task or with another architecture. To avoid this warning, please make sure the architecture of the model is correct in the config file (bert_config.json) and the weights are correctly initialized. This is NOT expected if you are initializing BertForQuestionAnswering from the checkpoint of a model that you expect to be exact.
 tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 3.58kB/s]
 vocab.txt: 100% 232k/232k [00:00<00:00, 4.07MB/s]
 tokenizer.json: 100% 466k/466k [00:00<00:00, 8.90MB/s]
 Context:
 BERT (Bidirectional Encoder Representations from Transformers) is a powerful pre-trained language model developed by Google. It
 has achieved state-of-the-art results on a wide range of NLP tasks, including Question Answering, Sentiment Analysis, Text
 Classification, and more.
 Question: Who developed BERT?
 Answer: google

Here's an example of how to use BERT for text classification:

```
from transformers import BertForSequenceClassification, BertTokenizer
import torch

# Load a pre-trained BERT model for sequence classification
# You can replace 'bert-base-uncased' with other BERT variants
model = BertForSequenceClassification.from_pretrained('bert-base-uncased')
```

```

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Example text
text = "This movie is amazing!"

# Tokenize the text and convert to input IDs and attention mask
inputs = tokenizer(text, return_tensors='pt')

# Make a prediction
with torch.no_grad():
    outputs = model(**inputs)

# The output 'logits' represent the scores for each class
# For a binary classification problem, there would be two logits
logits = outputs.logits

# To get the predicted class, you can take the argmax of the logits
predicted_class_id = torch.argmax(logits).item()

print(f"Input text: {text}")
print(f"Logits: {logits}")
print(f"Predicted class ID: {predicted_class_id}")

# Note: To get meaningful class labels, you would need to fine-tune the model
# on a labeled dataset specific to your classification task.

```

config.json: 100% 570/570 [00:00<00:00, 31.9kB/s]

model.safetensors: 100% 440M/440M [00:06<00:00, 57.0MB/s]

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.35kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 5.03MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 30.0MB/s]

Input text: This movie is amazing!
Logits: tensor([[0.1041, -0.1640]])
Predicted class ID: 0

Start coding or [generate](#) with AI.

```

from transformers import BertForTokenClassification, BertTokenizer
import torch

# Load a pre-trained BERT model and tokenizer for Named Entity Recognition
# 'dslim/bert-base-NER' is a fine-tuned BERT model for NER
model_ner = BertForTokenClassification.from_pretrained('dslim/bert-base-NER')
tokenizer_ner = BertTokenizer.from_pretrained('dslim/bert-base-NER')

# Example text
text_ner = "Angela Merkel is a politician in Germany."

# Tokenize the text
inputs_ner = tokenizer_ner(text_ner, return_tensors="pt")

# Get the predictions
with torch.no_grad():
    outputs_ner = model_ner(**inputs_ner)

# Get the predicted token labels
predictions_ner = torch.argmax(outputs_ner.logits, dim=2)

# Get the tokens and their corresponding predicted labels
tokens_ner = tokenizer_ner.convert_ids_to_tokens(inputs_ner["input_ids"][0])
predicted_labels_ner = [model_ner.config.id2label[t.item()] for t in predictions_ner[0]]

print("Named Entity Recognition (NER) Example:")
for token, label in zip(tokens_ner, predicted_labels_ner):
    print(f"{token}: {label}")

```

```

config.json: 100%                               829/829 [00:00<00:00, 18.0kB/s]

model.safetensors: 100%                         433M/433M [00:09<00:00, 82.5MB/s]
Some weights of the model checkpoint at dslim/bert-base-NER were not used when initializing BertForTokenClassification: ['bert
- This IS expected if you are initializing BertForTokenClassification from the checkpoint of a model trained on another task or
- This IS NOT expected if you are initializing BertForTokenClassification from the checkpoint of a model that you expect to be
tokenizer_config.json: 100%                      59.0/59.0 [00:00<00:00, 827B/s]

vocab.txt:      213k/? [00:00<00:00, 2.99MB/s]

added_tokens.json: 100%                        2.00/2.00 [00:00<00:00, 29.2B/s]

special_tokens_map.json: 100%                  112/112 [00:00<00:00, 2.24kB/s]
Named Entity Recognition (NER) Example:
[CLS]: O
Angela: B-PER
Me: I-PER
##rk: T-PER

```

```

from transformers import BertForSequenceClassification, BertTokenizer
import torch

# Load a pre-trained BERT model and tokenizer for Sentiment Analysis
# 'nlptown/bert-base-multilingual-uncased-sentiment' is a fine-tuned BERT model for sentiment analysis
model_sentiment = BertForSequenceClassification.from_pretrained('nlptown/bert-base-multilingual-uncased-sentiment')
tokenizer_sentiment = BertTokenizer.from_pretrained('nlptown/bert-base-multilingual-uncased-sentiment')

# Example text
text_sentiment = "I love this product, it's amazing!"

# Tokenize the text
inputs_sentiment = tokenizer_sentiment(text_sentiment, return_tensors="pt")

# Get the predictions (logits)
with torch.no_grad():
    outputs_sentiment = model_sentiment(**inputs_sentiment)

# Get the predicted class (sentiment score)
predicted_class_sentiment = torch.argmax(outputs_sentiment.logits, dim=1).item() + 1 # Add 1 because labels are 1-5

print("\nSentiment Analysis Example:")
print(f"Input text: {text_sentiment}")
print(f"Predicted sentiment score (1-5): {predicted_class_sentiment}")

# Note: The sentiment scores typically range from 1 to 5, where 1 is very negative
# and 5 is very positive, depending on the specific model used.

```

```

config.json: 100%                               953/953 [00:00<00:00, 77.4kB/s]

model.safetensors: 100%                         669M/669M [00:09<00:00, 79.6MB/s]

tokenizer_config.json: 100%                     39.0/39.0 [00:00<00:00, 2.02kB/s]

vocab.txt:      872k/? [00:00<00:00, 24.9MB/s]

special_tokens_map.json: 100%                  112/112 [00:00<00:00, 5.25kB/s]

Sentiment Analysis Example:
Input text: I love this product, it's amazing!
Predicted sentiment score (1-5): 5

```